
Informe de Trabajo Práctico 1

Estrategias - Mochila

Materia: Teoría de Algoritmos

Cátedra: Echevarría

Cuatrimestre: Segundo Cuatrimestre

Integrantes

Alumno, Padrón: Francisco Oscar Ballerio, 114777

Alumno, Padrón: ,

Alumno, Padrón: ,

Alumno, Padrón: ,

Fecha de Entrega: 3 de octubre de 2026

Índice

1. Definición Formal del Problema	2
2. Fuerza Bruta vs. Backtracking	2
3. Greedy	2
4. Programación Dinámica	2
4.1. Diseño del Algoritmo	2
4.1.1. Variante Alternativa del Problema	2
4.1.2. Principio de Optimalidad y Subestructura Óptima	2
4.1.3. Ecuación de Recurrencia	3
4.1.4. Ecuación de Recurrencia Alternativa	3
4.2. Pseudocódigo y Estructuras de Datos	3
4.2.1. Supuestos y Condiciones del Algoritmo	3
4.2.2. Estructuras de Datos	4
4.2.3. Pseudocódigo	4
4.3. Complejidad	5
4.3.1. Temporal	5
4.3.2. Espacial	5
4.4. Seguimiento del Problema Original	6
4.5. Comparación de Tiempos de Ejecución	6
4.5.1. Planteo Tradicional	6
4.5.2. Planteo Alternativo	7
4.5.3. Comparación y Conclusión	8

1. Definición Formal del Problema

Dado un límite de capacidad $W \in \mathbb{Z}_{\geq 0}$ y un conjunto de N elementos indexados por $i \in \{1, \dots, N\}$, donde cada elemento i posee un peso $w_i \in \mathbb{Z}^+$ y un valor asociado $v_i \in \mathbb{R}^+$, el objetivo consiste en seleccionar un subconjunto de elementos cuya suma de pesos no exceda W , maximizando el valor acumulado total.

2. Fuerza Bruta vs. Backtracking

3. Greedy

4. Programación Dinámica

4.1. Diseño del Algoritmo

4.1.1. Variante Alternativa del Problema

Junto con el problema original, incluimos su versión alternativa para esta estrategia. Dado un requerimiento de beneficio objetivo $B \in \mathbb{Z}_{\geq 0}$ y un conjunto de N elementos indexados por $i \in \{1, \dots, N\}$, donde cada elemento i posee un peso $w_i \in \mathbb{Z}^+$ y un valor o beneficio $v_i \in \mathbb{Z}^+$, el objetivo consiste en seleccionar un subconjunto de elementos cuya suma acumulada de beneficio alcance al menos B , minimizando el peso total resultante.

4.1.2. Principio de Optimalidad y Subestructura Óptima

Al procesar cada elemento i , el espacio de decisiones se reduce a dos alternativas mutuamente excluyentes:

- **Excluir el elemento i :** La ganancia obtenida equivale a la solución óptima calculada para el prefijo de los primeros $i - 1$ elementos disponiendo de la misma capacidad w .
- **Incluir el elemento i :** Opción admisible únicamente si $w_i \leq w$. El valor resultante corresponde a sumar v_i a la solución óptima del subproblema que considera los primeros $i - 1$ elementos con la capacidad residual $w - w_i$.

Variante alternativa:

- **Excluir el elemento i :** No se agrega peso adicional ni se aporta beneficio. El costo equivale al peso mínimo requerido para alcanzar la cuota b empleando únicamente los primeros $i - 1$ elementos.
- **Incluir el elemento i :** Se suma el peso w_i al costo total y se cubren v_i unidades del beneficio demandado. El subproblema restante consiste en hallar el peso mínimo necesario para cubrir el remanente $\max(0, b - v_i)$ considerando los primeros $i - 1$ elementos.

La validez de este planteo descansa en la propiedad de **subestructura óptima** (Principio de Optimalidad de Bellman): la solución óptima global contiene necesariamente soluciones óptimas a los subproblemas que la componen. Al parametrizar el espacio de estados en dos dimensiones (i, w) , la tabla de programación dinámica evalúa y almacena sistemáticamente las soluciones parciales óptimas, desacoplando la decisión sobre el elemento i de la resolución interna de los estados anteriores.

4.1.3. Ecuación de Recurrencia

Definiendo $\text{OPT}(i, w)$ como el valor óptimo alcanzable considerando un subconjunto de los primeros i elementos con capacidad máxima disponible w :

$$\text{OPT}(i, w) = \begin{cases} 0 & \text{si } i = 0 \vee w = 0, \\ \text{OPT}(i - 1, w) & \text{si } w_i > w, \\ \max \left(\text{OPT}(i - 1, w), v_i + \text{OPT}(i - 1, w - w_i) \right) & \text{si } w_i \leq w. \end{cases} \quad (1)$$

El valor de la solución óptima global se ubica en $\text{OPT}(N, W)$.

4.1.4. Ecuación de Recurrencia Alternativa

Definiendo $\text{OPT}(i, b)$ como el peso mínimo requerido para alcanzar al menos un beneficio b empleando un subconjunto de los primeros i elementos:

$$\text{OPT}(i, b) = \begin{cases} 0 & \text{si } b \leq 0, \\ \infty & \text{si } i = 0 \wedge b > 0, \\ \min \left(\text{OPT}(i - 1, b), w_i + \text{OPT}(i - 1, \max(0, b - v_i)) \right) & \text{en otro caso.} \end{cases} \quad (2)$$

El valor infinito modela un estado infactible: si la cantidad de elementos disponibles es nula ($i = 0$) pero el beneficio demandado es estrictamente positivo ($b > 0$), no existe combinación posible para cubrirlo. Asignar infinito evita que este caso base sea seleccionado al calcular los mínimos en las transiciones subsiguientes.

4.2. Pseudocódigo y Estructuras de Datos

La solución a cada problema está compuesta por una única función: `mochilaPd` y `mochilaPdAlt`.

- `mochilaPd` y `mochilaPdAlt` reciben como parámetros una lista de valores V , una lista de pesos P , la cantidad de elementos N y la capacidad de la mochila W o el beneficio a alcanzar B , respectivamente.

4.2.1. Supuestos y Condiciones del Algoritmo

Para que el algoritmo funcione correctamente, se asume:

- $\forall i \in \{0, \dots, N - 1\}$, $V[i]$ y $P[i]$ corresponden al mismo objeto.

- $\forall i \in \{0, \dots, N-1\}, V[i] \geq 1$ y $P[i] \geq 1$.
- La cantidad de elementos en V y P es igual a N .

mochilaPd:

- Encuentra el mayor beneficio posible sin sobrepasar la capacidad W de la mochila, según los valores V y los pesos P .

mochilaPdAlt:

- Encuentra el menor peso posible para obtener al menos el beneficio B , según los valores V y los pesos P .

4.2.2. Estructuras de Datos

- **Lista de listas (matriz):** Se utiliza una estructura bidimensional para calcular y almacenar progresivamente los resultados óptimos de los subproblemas.

4.2.3. Pseudocódigo

```
def mochilaPd(valores , pesos , cant_elementos , capacidad_mochila):
    optimos = []
    para i desde 0 hasta cant_elementos:
        agrego a optimos lista de tamaño (capacidad_mochila + 1) con cero

    para i desde 1 hasta cant_elementos:
        v_actual = valores[i - 1]
        p_actual = pesos[i - 1]

        para w desde 1 hasta capacidad_mochila:
            if p_actual <= w:
                optimos[i][w] = max(
                    v_actual + optimos[i - 1][w - p_actual],
                    optimos[i - 1][w]
                )
            else:
                optimos[i][w] = optimos[i - 1][w]

    devuelvo optimos[cant_elementos][capacidad_mochila]

def mochilaPdAlt(valores , pesos , cant_elementos , beneficio_min):
    optimos = []
    para i desde 0 hasta cant_elementos:
        agrego a optimos lista de tamaño (beneficio_min + 1) con INF

    para i desde 0 hasta cant_elementos:
        optimos[i][0] = 0

    para i desde 1 hasta cant_elementos:
```

```

v_actual = valores[i - 1]
p_actual = pesos[i - 1]

para b desde 1 hasta beneficio_min:
    beneficio_restante = max(0, b - v_actual)
    optimos[i][b] = min(
        optimos[i - 1][b],
        p_actual + optimos[i - 1][beneficio_restante]
    )

devuelvo optimos[cant_elementos][beneficio_min]

```

4.3. Complejidad

4.3.1. Temporal

La complejidad temporal de un algoritmo se mide según el tamaño de su entrada y el peor caso representable con ella.

El tamaño de la entrada se define según la cantidad de bits que ocupa en memoria.

mochilaPd: itera sobre todos los elementos y sobre todas las posibles capacidades enteras de la mochila desde 0 hasta W . La complejidad final es $O(N \cdot W)$.

mochilaPdAlt: itera sobre todos los elementos y sobre todos los posibles beneficios enteros desde 0 hasta B . La complejidad final es $O(N \cdot B)$.

En ambos casos N está determinado por la cantidad de elementos de entrada. W y B deben ser representados por una tira de bits. Para almacenarlos en memoria se requieren $k = \lceil \log_2(W + 1) \rceil \approx \lceil \log_2(B + 1) \rceil$ bits, por lo que $W, B \approx 2^k$.

Finalmente, la complejidad de ambos algoritmos es pseudopolinomial, dado que al incrementar en uno los bits necesarios para representar W o B , la cantidad de operaciones que efectúa el algoritmo se duplica (2^{k+1}), definiendo:

$$O(N \cdot 2^k)$$

4.3.2. Espacial

La complejidad espacial cuantifica la memoria auxiliar requerida durante la ejecución en función del tamaño de la entrada.

mochilaPd: almacena una tabla de programación dinámica de dimensiones $(N + 1) \times (W + 1)$ para registrar los resultados de los subproblemas y posibilitar la reconstrucción de la solución, resultando en una complejidad de $O(N \cdot W)$ (reducible a $O(W)$ si se optimiza el espacio manteniendo únicamente la fila anterior cuando solo se busca el valor máximo).

mochilaPdAlt: requiere una matriz de dimensiones $(N + 1) \times (B + 1)$ para registrar el peso mínimo asociado a cada beneficio posible, demandando una complejidad de

$O(N \cdot B)$ (u $O(B)$ en caso de optimizar memoria).

Al igual que en el análisis temporal, el espacio utilizado depende del valor numérico de W o B . Al representarse en memoria con k bits ($W, B \approx 2^k$), la complejidad espacial de la matriz completa escala con dicho valor y resulta pseudopolinomial:

$$O(N \cdot 2^k)$$

4.4. Seguimiento del Problema Original

Indice	Valores	Pesos	Cantidad de elementos N ↓	Capacidad de la mochila W →					
0	0	0		↓ n / w →	0	1	2	3	4
1	40	3		0	0	0	0	0	0
2	25	2		1	0	0	0	40	40
3	25	2		2	0	0	25	40	40
4	29	3		3	0	0	25	40	50
				4	0	0	0	40	50

Figura 1: Seguimiento en Google Sheets.

En la primera columna todo es cero, pues con capacidad cero el valor es cero; de manera análoga, en la primera fila con cero elementos el beneficio acumulado es cero. A continuación, a cada una de las celdas se le aplica la ecuación de recurrencia previamente presentada para alcanzar el resultado óptimo final.

4.5. Comparación de Tiempos de Ejecución

En la teoría conocemos la complejidad de ambos algoritmos. Ahora buscamos comprobar empíricamente que estas complejidades sean correctas. Por lo tanto, usaremos el método de aproximación por cuadrados mínimos para verificar que el error residual sea relativamente bajo. Los valores evaluados abarcan hasta $N = 1500$ debido a la complejidad espacial del algoritmo.

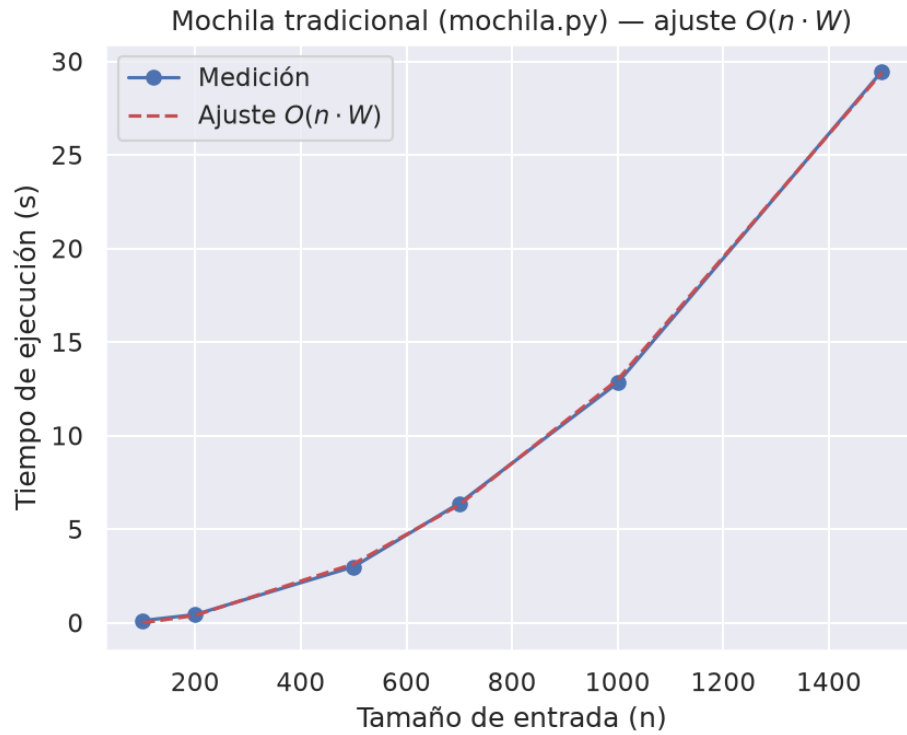
4.5.1. Planteo Tradicional

Tabla 1: Tiempos de ejecución medidos para la versión tradicional (`mochila.py`).

N	Capacidad W	Tiempo promedio (s)
100	5000	0.1147
200	10000	0.4375
500	25000	2.9937
700	35000	6.3836
1000	50000	12.8195
1500	75000	29.4668

Tabla 2: Ajuste por cuadrados mínimos (`mochila.py`).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
<code>mochila_tradicional</code>	$c_1(n \cdot W) + c_2$	$2,62 \times 10^{-7}$	$-1,34 \times 10^{-1}$	$8,12 \times 10^{-2}$

Figura 2: Mochila tradicional (`mochila.py`) — ajuste $O(n \cdot W)$.

4.5.2. Planteo Alternativo

Tabla 3: Tiempos de ejecución medidos para la versión alternativa (`mochila_alt.py`).

N	Beneficio B	Tiempo promedio (s)
100	5000	0.1459
200	10000	0.5531
500	25000	3.8525
700	35000	7.9058
1000	50000	16.2200
1500	75000	37.8726

Tabla 4: Ajuste por cuadrados mínimos (`mochila_alt.py`).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
<code>mochila_alternativo</code>	$c_1(n \cdot B) + c_2$	$3,37 \times 10^{-7}$	$-2,61 \times 10^{-1}$	$2,79 \times 10^{-1}$

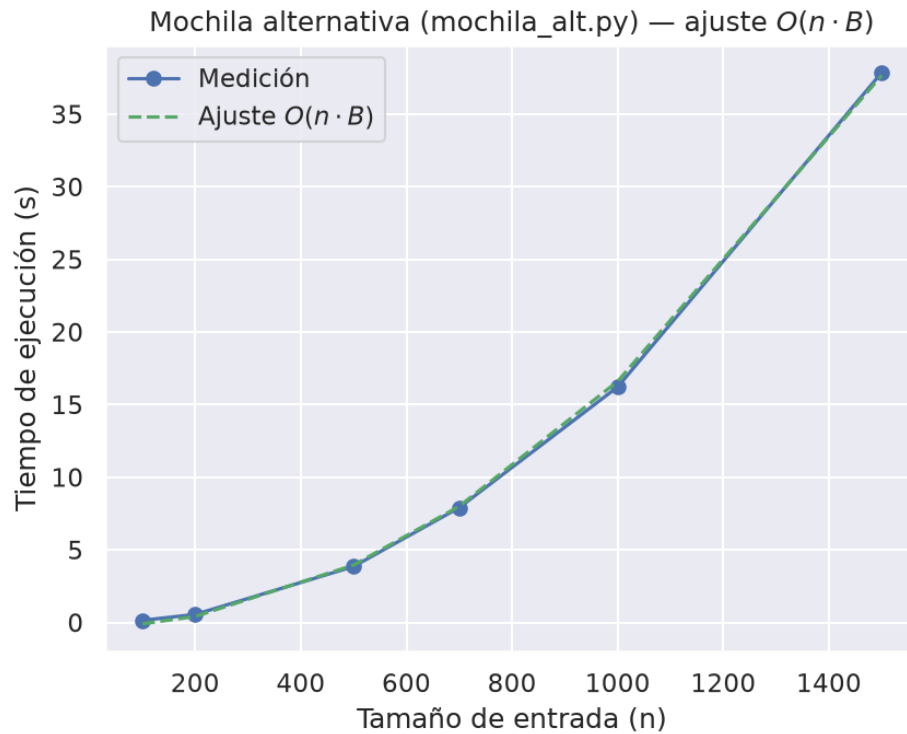


Figura 3: Mochila alternativa (mochila_alt.py) — ajuste $O(n \cdot B)$.

4.5.3. Comparación y Conclusión

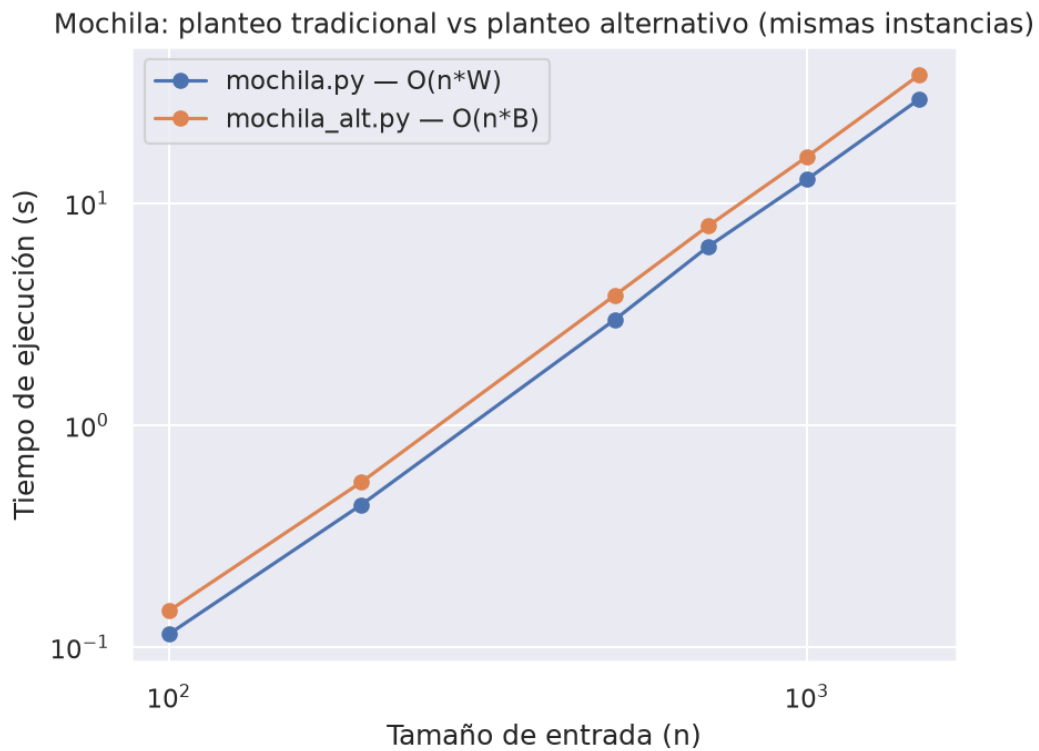


Figura 4: Mochila: planteo tradicional vs. planteo alternativo (mismas instancias, escala log-log).

En los experimentos realizados, tanto la capacidad W como la cota de beneficio B escalan linealmente con el tamaño de entrada ($W = B = 50 \cdot N$). Como consecuencia, la complejidad teórica $O(n \cdot W)$ y $O(n \cdot B)$ se traduce en un crecimiento cuadrático respecto a n , lo cual se evidencia en la curvatura de las curvas individuales y en la pendiente cercana a 2 en la escala log-log.

Al comparar ambos algoritmos sobre las mismas instancias ($W = B$), el planteo tradicional (`mochila.py`) resulta aproximadamente un 27 % a 28 % más rápido que el planteo alternativo (`mochila_alt.py`), reflejado en una constante oculta menor ($c_1 \approx 2,62 \times 10^{-7}$ frente a $3,37 \times 10^{-7}$). Ambos modelos presentan un error cuadrático residual bajo (0,0812 y 0,2786), lo que valida experimentalmente los órdenes de complejidad teóricos planteados. En la práctica, el planteo tradicional es preferible cuando $W \ll B$, mientras que el alternativo resulta ventajoso cuando el beneficio total acotado es menor a la capacidad ($B \ll W$).